PRODUCTION DEPLOYMENT RUNBOOK

School ERP
Production Deployment Guide

A practical, project-specific procedure for deploying and re-deploying the `rnd-service` API and `rnd-ui` frontend to a secure, HTTPS, single-server production environment.

Backend	school-erp-backend · Node 20 · Express 4 · TypeScript · MongoDB · Redis · BullMQ · Socket.IO
Frontend	client (rnd-ui) · React 19 · Vite 8 · TailwindCSS · static SPA
Repos	github.com/team-IIT/rnd-service · github.com/team-IIT/rnd-ui
Target	1× AWS EC2 (Ubuntu 24.04) + Nginx + PM2 + MongoDB Atlas + AWS S3 + Meta WhatsApp Cloud API
Document date	2026-08-27
Status	Analysis + fixes documented. No application code was modified by this guide — all required edits are specified in §5.

Table of Contents

1. Executive Deployment Overview
2. Current Project Architecture (Assessment)
3. Problems Found
4. Target Production Architecture
5. Required Code & Config Changes (before deploy)
6. Prerequisites
7. Environment Variable Configuration
8. AWS Server Configuration
9. Domain / DNS Configuration
10. Database Configuration (MongoDB Atlas)
11. Backend Deployment
12. Frontend Deployment
13. Nginx Configuration
14. HTTPS / SSL Configuration
15. PM2 / Process Management
16. CORS Configuration
17. API Routing Configuration
18. Authentication Configuration
19. File Upload Configuration
20. WhatsApp Cloud API & Webhook
21. Security Hardening
22. Health Checks
23. Logging & Monitoring
24. Production Testing Checklist
25. Troubleshooting Guide
26. Rollback Procedure
27. Backup & Recovery
28. Future Deployment / Update Procedure
29. Multi-School Deployment Considerations
30. Future CI/CD Automation
31. One-Page Deployment Runbook
32. Running List of Changes

1. Executive Deployment Overview

This ERP is a two-part application: a stateful Node/Express API (`rnd-service`) and a static React SPA (`rnd-ui`). Both run comfortably on a **single small EC2 instance**. MongoDB is external (Atlas), object storage is external (S3), and WhatsApp is external (Meta Graph API).

Chosen strategy — and why

Decision	Choice	Reason
Compute	1× AWS EC2 `t3.small` (2 GB), Ubuntu 24.04 LTS	API + BullMQ worker + Redis + Nginx fit one box. Native deps (`sharp` , `libvips` , `ffmpeg-static` , `@imgly/background-removal-node`) make the free-tier 1 GB tight; 2 GB (or 1 GB + 2 GB swap) is the safe minimum.
Process manager	PM2 (host), not Docker	`ecosystem.config.js` and the `pm2` dependency already exist. Host PM2 gives the simplest logs/restart story and lowest memory overhead. Docker Compose remains a fully supported alternative (files already in the repo — see §11.9).
Web tier	Host Nginx as the single public entrypoint	Serves the SPA from disk *and* reverse-proxies `/api` , `/socket.io` , `/healthz` to `127.0.0.1:5000` . TLS terminates here.
Domain layout	**Single domain, path-based**: `https://erp.<school>.com` serves the app; `…/api` is the backend	Frontend and API are same-origin. This makes CORS a non-issue and, critically, keeps the `SameSite=strict` refresh-token cookie working (a subdomain split would silently break session refresh — see §18).
Database	MongoDB Atlas (M0 to start, M10 for real load)	No Mongo on the box to back up, patch, or run out of disk. Atlas IP-allowlists the EC2 Elastic IP.
Orchestration	None (no Kubernetes, no ECS)	Single-tenant, single-node workload. Extra infrastructure would add cost and failure modes with no benefit.
TLS	Let's Encrypt via Certbot (`--nginx`), auto-renew	Free, trusted, and the systemd timer handles renewal.

> **TIME BUDGET**
> First deployment: ~90 minutes (30 of which is DNS/Atlas propagation). Subsequent deployments: one `./deploy.sh` , ~2 minutes.

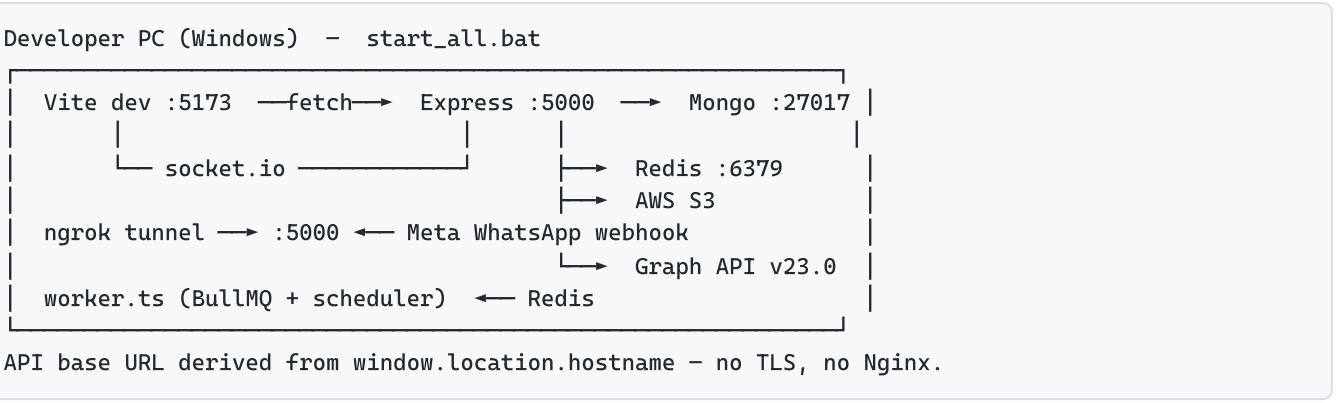
2. Current Project Architecture (Assessment)

2.1 What exists today

Concern	Finding (from code inspection)
Frontend framework	React 19 + Vite 8, TypeScript, TailwindCSS 3. Package name `client`. **No React Router** — views are switched from `window.location.pathname` and query-string tokens in src/App.tsx.
Backend framework	Express 4.19 on Node (Dockerfile pins **node:20-slim**), TypeScript 5.4 compiled with `tsc` → `dist/`. Entry src/server.ts; separate worker entry src/worker.ts.
Language / package manager	TypeScript / **npm** (`package-lock.json` committed in both repos).
Build	Backend: `npm run build` = `tsc`. Frontend: `npm run build` = `vite build` → `dist/` static assets.
Start commands	Backend API: `node dist/server.js`. Worker: `node dist/worker.js`. (Dev-only: `npm run dev` / `dev:worker` via ts-node-dev — **not for production**.)
API structure	All under `/api/*`. Public: `/api/auth/*`, `/api/receipts`, `/api/whatsapp/*`, `/api/school-profile`, `/api/report-card/verify`. Everything else is `authenticateJWT`-guarded. Also non-API: `/`, `/healthz`, `/metrics`, `/superadmin`.
Authentication	Passport (local + JWT). Access token JWT (1h) returned in JSON, held **in memory** by the SPA, sent as `Authorization: Bearer`. Refresh token JWT (7d) in an **HttpOnly cookie** `refreshToken` (`SameSite=strict`, `Secure` in production). Rotation on `POST /api/auth/refresh`. Sessions tracked in Redis with a MongoDB fallback (refreshToken.model.ts). Passwords hashed with bcryptjs (model pre-save hook).
Database / ODM	MongoDB via Mongoose 8 (src/config/db.ts, pool 10–50). On boot it auto-seeds default classes, the **permission catalog**, and a **superadmin user** (see §3, issue D).
Cache / queue	Redis via ioredis (src/config/redis.ts). BullMQ queues (src/config/queue.ts): `backupQueue`, `notificationQueue`, `assetOptimizationQueue`. Consumed by the worker process (src/config/worker.ts). Producers fall back to synchronous execution if Redis is down.
File uploads	Base64 payloads inside JSON bodies (`express.json({ limit: '250mb' })`), pushed to **AWS S3** (`@aws-sdk/client-s3`, presigned download URLs). Small message attachments (<10 MB) fall back to a local `uploads/` folder served at `/uploads` — **ephemeral, avoid in production**.
WebSockets	Socket.IO 4.8 on the same HTTP server (src/config/socket.ts). JWT handshake auth (token in `auth` / `query`). Rooms per `user:` / `role:` / `student:` / `teacher:`. Client: src/context/socketContext.tsx connects to `API_BASE` minus `/api`.
Background jobs	BullMQ workers (backups → S3 CSV, WhatsApp send retries, asset optimisation) + a DB scheduler (src/services/scheduler.ts) for reminders / overdue markers, started in src/worker.ts.
Email	**Removed.** `src/services/email/*` is deleted on this branch. Password reset is now WhatsApp-OTP only.
Third-party APIs	Meta WhatsApp Cloud API (Graph `v23.0`) — messaging, templates, and an inbound webhook. AWS S3. No AI API keys in use (image background-removal runs locally via `@imgly/background-removal-node`).
Existing deploy files	Dockerfile (multi-stage, `pm2-runtime`), docker-compose.yml (backend + redis + nginx; expects external Atlas + mounted certs), nginx/nginx.conf (API-only reverse proxy, HTTP only, HTTPS block commented out), ecosystem.config.js (PM2: `school-erp-backend` + `school-erp-worker`, fork mode, 1 instance, 1 GB restart), start_all.bat (Windows dev launcher with ngrok — dev only), .env.example (good, placeholders only).

Frontend API config	src/constants/index.ts: `API_BASE = VITE_API_URL || `${protocol}//${hostname}:5000/api`` . `VITE_API_URL` is intentionally unset for dev.
CORS config	src/server.ts: allow-list = dev origins + `CORS_ALLOWED_ORIGINS` (CSV) + any private-LAN origin, `credentials:true` . Socket.IO CORS is `origin:'*'` (broken combo — see §3).
Ports	Backend `5000` , Vite dev `5173` , Redis `6379` , Mongo `27017` .

2.2 Current architecture diagram (development)



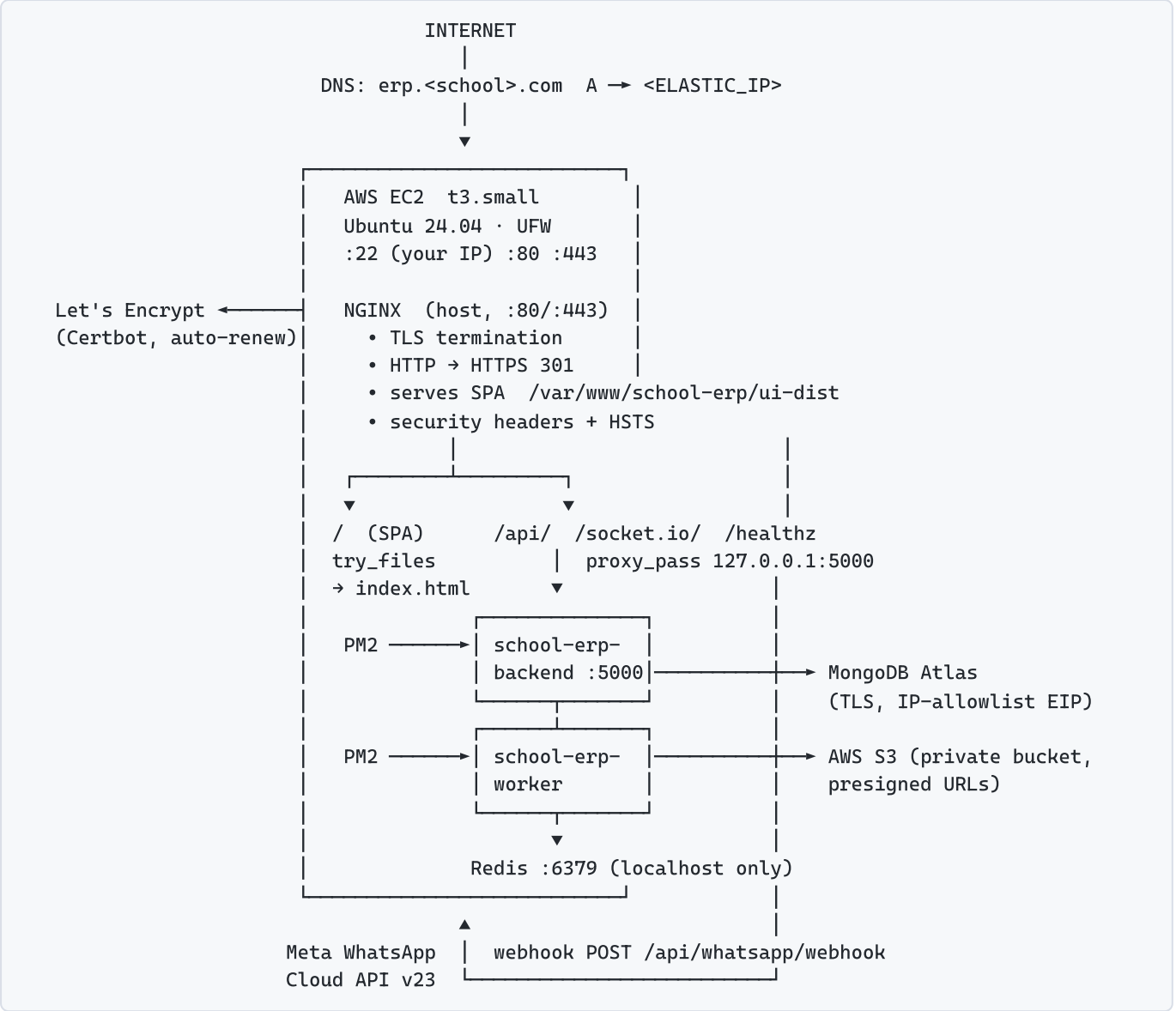
3. Problems Found

Every issue below is deployment-relevant. Severity: BLOCKER must be fixed before go-live, HIGH fix during deployment, MED hardening.

#	Sev	Problem	Evidence	Fix (detail in §)
A	BLOCKER	**No `trust proxy`.** Behind Nginx, `req.ip` is always `127.0.0.1`, so the Redis-backed login rate-limiter keys every user into one bucket (one attacker locks out the whole school after 20 tries), and `express-rate-limit` v8 logs a misconfiguration validation error. `req.ip` is also stored on every refresh-token record.	src/server.ts — no `app.set('trust proxy', …)` anywhere.	§5.1
B	BLOCKER	**Frontend `VITE_API_URL` not set for production.** Without it the built bundle calls `https://erp.example.com:5000/api` — port 5000 is firewalled, so the app cannot reach the API at all.	rnd-ui/src/constants/index.ts:3-7	§7, §12
C	HIGH	**Socket.IO CORS is `origin:'*'` with `credentials:true`** — an invalid combination browsers reject, and unrestricted. Also does not reflect the production origin.	src/config/socket.ts:11-16	§5.2
D	HIGH	**Auto-seeded superadmin with password `123`.** On any boot where no `superadmin` exists, src/config/db.ts creates `username: superadmin / password: 123` and does *not* set `mustChangePassword`. That is a public credential to the highest-privilege account.	src/config/db.ts:73-91	§5.3, §18.4
E	HIGH	**Verification links hardcode `http://`.** Fee-receipt and report-card links sent to parents over WhatsApp are built as `http://${APP_HOST}/?verify_token=…`. Over a HTTPS deployment this is an insecure scheme (browser upgrade/mixed-content, looks untrustworthy). Same for the `/superadmin` redirect.	dashboard.routes.ts:439, reportCard.service.ts:331, server.ts:179	§5.4
F	HIGH	**`GET /metrics` is unauthenticated** and returns Prometheus process/DB metrics to anyone on the internet.	src/server.ts:202	§5.6 / §13 (Nginx `deny`)
G	HIGH	**Existing `nginx/nginx.conf` is not usable as-is**: it proxies *everything* to `backend:5000` (Docker service name), never serves the SPA, has no SPA fallback, no TLS (commented out), `server_name localhost`, and `client_max_body_size 50M` while Express accepts 250 MB.	nginx/nginx.conf	§13 (full replacement)
H	MED	**`.env.production` is not gitignored.** `rnd-service/.gitignore` ignores `.env`, `.env.local`, `.env.*.local` — but a file literally named `.env.production` would be committed. (`.env` itself has never been committed — verified.)	rnd-service/.gitignore:8-11	§5.5
I	MED	**No `helmet` / security headers** from the app. No HSTS, X-Frame-Options, nosniff, etc.	`package.json` — no `helmet`.	§13 (Nginx adds them) / §21

J	MED	**250 MB JSON body limit** is a memory-exhaustion vector (base64 inflates ~33%; a few concurrent large POSTs can OOM a 2 GB box).	src/server.ts:100	§19
K	MED	**WhatsApp webhook accepts unsigned payloads** unless `WHATSAPP_APP_SECRET` is set. In production the code now rejects all deliveries without it — so it *must* be configured or inbound status/replies silently stop.	whatsappWebhook.routes.ts:46-74	§20
L	MED	**Redis is a hard dependency for login.** The rate-limiter store calls Redis directly; if Redis is down, `POST /api/auth/login` errors (500) for everyone. Redis must be running and monitored.	src/server.ts:115-126	§15, §22
M	MED	**Local `uploads/` fallback is ephemeral.** If `AWS_BUCKET_NAME` is unset, small attachments land on the box and are lost on redeploy; the backup job hard-fails. S3 must be configured.	message.routes.ts:37, config/worker.ts:19	§19
N	LOW	**Dev origins always allowed by CORS** (`localhost:5173/5174` and any private-LAN IP) even in production. Low risk behind a single public domain, but unnecessary surface.	src/server.ts:77-96	§16 (optional)
O	LOW	**`/healthz` returns HTTP 200 even when MongoDB is down** (body `status:"degraded"`). Uptime monitors must inspect the body, not just the status code.	src/server.ts:187-199	§22

4. Target Production Architecture



Key properties: one public domain; SPA and API same-origin (no CORS, cookies work); only 22/80/443 exposed; Redis and the Node port bound to localhost; database and object storage are managed services.

5. Required Code & Config Changes (before deploy)

> **APPLY THESE ON A BRANCH, BUILD, AND COMMIT BEFORE DEPLOYING.**
> None of the app's business logic changes. Each edit below is small and isolated.

5.1 rnd-service/src/server.ts — trust the Nginx proxy BLOCKER

Immediately after `const app = express();` (line 67):

```
const app = express();

// Behind exactly one reverse proxy (Nginx). Makes req.ip / X-Forwarded-For
// correct so per-IP login rate limiting and refresh-token audit rows work.
app.set('trust proxy', 1);
```

5.2 rnd-service/src/config/socket.ts — real CORS allow-list HIGH

Replace the `new SocketIOServer(...)` block (lines 10-16):

```
const DEV_ORIGINS = ['http://localhost:5173', 'http://localhost:5174'];
const allowedOrigins = [
  ...DEV_ORIGINS,
  ...(process.env.CORS_ALLOWED_ORIGINS || '')
    .split(',').map((o) => o.trim()).filter(Boolean),
];

io = new SocketIOServer(server, {
  cors: { origin: allowedOrigins, credentials: true },
});
```

(Same-origin path routing means the handshake is same-origin and CORS is not exercised, but this removes the invalid `*` + `credentials` pair and is correct if a subdomain split is ever adopted.)

5.3 rnd-service/src/config/db.ts — do not seed a known superadmin password HIGH

In the "Auto-seed default superadmin" block (~line 77), make the password come from the environment and force a change on first login:

```
const seedPass = process.env.SUPERADMIN_INITIAL_PASSWORD;
if (!superAdminExists && seedPass) {
  await User.create({
    username: 'superadmin',
    password: seedPass,
    email: process.env.SUPERADMIN_EMAIL || 'superadmin@schoolhub.edu',
    role: 'superadmin',
    firstName: 'Super', lastName: 'Admin',
    phone: process.env.SUPERADMIN_PHONE || '+919999999990',
    isActive: true,
    mustChangePassword: true,
  });
}
```

Then set `SUPERADMIN_INITIAL_PASSWORD` (a long random value, letter+digit, ≥6 chars) in `.env` for the very first boot, and remove it afterwards. *Minimum acceptable alternative if you do not change the code:* on first

boot, log in as `superadmin / 123` and immediately change the password (§18.4), then confirm no other `superadmin` can be created.

5.4 Verification-link scheme → HTTPS HIGH

Three spots build parent-facing links with a hardcoded `http://`. Change each to `https://` (`APP_HOST` stays host-only, e.g. `erp.school.com`):

File	Change
src/routes/admin/dashboard.routes.ts:439	`` `http://${appHost}/?verify_token=…` `` → `` `https://${appHost}/?verify_token=…` ``
src/services/reportCard.service.ts:331	`` `http://${appHost}/?report_verify_token=…` `` → `` `https://${appHost}/…` ``
src/server.ts:179	`res.redirect(`http://${process.env.APP_HOST \|\| 'localhost:5173'}/super-admin`)` → use `https://` when `APP_HOST` is not localhost, e.g.: `const h = process.env.APP_HOST \|\| 'localhost:5173'; res.redirect(`${h.startsWith('localhost') ? 'http' : 'https'}://${h}/super-admin`);`

5.5 rnd-service/.gitignore — never commit any real env file MED

```
# Environment files
.env
.env.*
!.env.example
```

Also add `nginx/certs/*` (keep `!nginx/certs/.gitkeep`) if you will ever place certs there for the Docker path.

5.6 rnd-service/src/server.ts — (optional) auth-gate `/metrics` MED

Either restrict it in Nginx (§13, simplest — `location = /metrics { return 404; }`) *or* wrap the handler: `app.get('/metrics', authenticateJWT, requireRole(['superadmin']), metricsHandler);`. Do one; Nginx is enough.

5.7 (Optional) reduce JSON body limit MED

src/server.ts:100 — `limit: '250mb'` → `limit: '60mb'`, matching the Nginx `client_max_body_size 60m`. Large media is already required to go to S3 (>10 MB threshold in message.routes.ts:34). Verify Events/Backups still work with 60 MB before shipping.

5.8 Build & commit

```
cd rnd-service
git checkout -b prod-hardening
# apply 5.1–5.7
npm run build          # must compile clean
npm test               # vitest — should stay green
git commit -am "prod: trust proxy, socket CORS, https links, env hygiene, metrics guard"
git push -u origin prod-hardening
# merge to main via PR, then deploy main
```

6. Prerequisites

Item	Needed
AWS account	EC2 + Elastic IP + IAM + an S3 bucket
Domain	A registrar login where you can add an **A record** for `erp.<school>.com`
MongoDB Atlas	Free/paid project; ability to create a DB user and edit Network Access
Meta WhatsApp	A WhatsApp Business App with: permanent System-User access token, Phone Number ID, WABA ID, App Secret, and a webhook verify token you invent
SSH keypair	An EC2 keypair (`.pem`) downloaded locally
Local tools	`git` , an SSH client; that's all — building happens on the server
Secrets generated	`openssl rand -hex 32` ×2 for `JWT_SECRET` and `JWT_REFRESH_SECRET` (must differ)

NAMING USED THROUGHOUT

Replace these placeholders everywhere: `<DOMAIN>` = `erp.school.com` · `<ELASTIC_IP>` = your EC2 public IP · `<SSH_KEY>` = path to your `.pem` · `<EC2_USER>` = `ubuntu` .

7. Environment Variable Configuration

7.1 Classification

Class	Variables	Where it lives
Backend server-only secrets	`JWT_SECRET`, `JWT_REFRESH_SECRET`, `MONGODB_URI`, `AWS_SECRET_ACCESS_KEY`, `WHATSAPP_ACCESS_TOKEN`, `WHATSAPP_APP_SECRET`, `SUPERADMIN_INITIAL_PASSWORD`	`/var/www/school-erp/api/.env` on the EC2 box only, `chmod 600`. Never in git, never sent to the browser.
Backend non-secret config	`NODE_ENV`, `PORT`, `APP_HOST`, `CORS_ALLOWED_ORIGINS`, `REDIS_HOST`, `REDIS_PORT`, `AWS_REGION`, `AWS_BUCKET_NAME`, `AWS_ACCESS_KEY_ID`, `WHATSAPP_PHONE_NUMBER_ID`, `WHATSAPP_BUSINESS_ACCOUNT_ID`, `WHATSAPP_WEBHOOK_VERIFY_TOKEN`	Same `.env` file.
Frontend build-time / **public**	`VITE_API_URL`	`/var/www/school-erp/ui/.env.production`. **Baked into the JS bundle at `vite build` — visible to anyone.** Must contain no secret.
Database secrets	Atlas DB username + password (inside `MONGODB_URI`)	Backend `.env` only.

FRONTEND EXPOSURE RULE

Vite only exposes `VITE_*` variables, and it exposes **all** of them into the public bundle. The only `VITE_*` var this project uses is `VITE_API_URL`. Never add a `VITE_` secret.

7.2 Backend `.env` (production) — copy from `.env.example` and fill

```
NODE_ENV=production
PORT=5000

# Host of the SPA (NO scheme). Used for parent-facing verify links + /superadmin redirect.
APP_HOST=<DOMAIN>

# Browser origins allowed through CORS + Socket.IO. Same-origin means this is
# belt-and-braces, but set it correctly.
CORS_ALLOWED_ORIGINS=https://<DOMAIN>

# ── Database ──
MONGODB_URI=mongodb+srv://erpUser:<DB_PASSWORD>@<cluster>.mongodb.net/school-erp?
retryWrites=true&w=majority&appName=school-erp

# ── Redis (local) ──
REDIS_HOST=127.0.0.1
REDIS_PORT=6379

# ── Auth (openssl rand -hex 32, twice, must differ) ──
JWT_SECRET=<64_HEX_CHARS>
JWT_REFRESH_SECRET=<DIFFERENT_64_HEX_CHARS>

# ── First-boot superadmin (see §5.3); remove after first successful boot ──
SUPERADMIN_INITIAL_PASSWORD=<random_letters_and_digits>
SUPERADMIN_PHONE=+91XXXXXXXXXX

# ── AWS S3 ──
AWS_ACCESS_KEY_ID=<iam_key>
AWS_SECRET_ACCESS_KEY=<iam_secret>
AWS_REGION=ap-southeast-2
AWS_BUCKET_NAME=<bucket>

# ── WhatsApp Cloud API (leave ALL blank for simulation mode) ──
WHATSAPP_ACCESS_TOKEN=<permanent_system_user_token>
WHATSAPP_PHONE_NUMBER_ID=<numeric>
WHATSAPP_BUSINESS_ACCOUNT_ID=<numeric>
WHATSAPP_WEBHOOK_VERIFY_TOKEN=<invent_a_long_random_string>
WHATSAPP_APP_SECRET=<meta_app_secret>
WHATSAPP_DEFAULT_COUNTRY_CODE=91
```

STARTUP FAIL-FAST

With `NODE_ENV=production` the server **refuses to boot** if `JWT_SECRET` / `JWT_REFRESH_SECRET` are unset or equal to the repo's dev defaults (config/secrets.ts), and if the `WHATSAPP_*` set is *partially* filled (config/whatsapp.ts). Either fill them all or leave them all blank.

7.3 Frontend `.env.production`

```
VITE_API_URL=https://<DOMAIN>/api
```

8. AWS Server Configuration

8.1 EC2 instance

Setting	Value
AMI	Ubuntu Server 24.04 LTS (x86_64)
Type	`t3.small` (2 vCPU / 2 GB). `t3.micro` only if you add 2 GB swap (§11.2).
Storage	30 GB gp3
Key pair	your `<SSH_KEY>`
Elastic IP	Allocate one and associate it — the DNS A record points here and it must not change on reboot.

8.2 Security Group (inbound)

Port	Source	Purpose
22 (SSH)	**Your IP /32 only**	Admin. Never `0.0.0.0/0`.
80 (HTTP)	`0.0.0.0/0`	ACME challenge + HTTPS redirect
443 (HTTPS)	`0.0.0.0/0`	The application

Do **not** open 5000, 6379, or 27017. Outbound: leave default (all) — needed for Atlas, S3, Meta, apt, npm.

8.3 IAM — least-privilege S3 access

Create an IAM user `school-erp-s3` (programmatic access), attach an inline policy scoped to the one bucket:

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Effect": "Allow",
      "Action": ["s3:PutObject","s3:GetObject","s3:DeleteObject","s3:ListBucket"],
      "Resource": ["arn:aws:s3:::<bucket>","arn:aws:s3:::<bucket>/*"] }
  ]
}
```

Use its access key/secret in `.env`. (An EC2 instance role is cleaner still, but the SDK reads env creds fine and it keeps this guide portable.)

8.4 S3 bucket

- Region = `AWS_REGION` (default in code: `ap-southeast-2`).
- **Block all public access: ON.** The app serves objects via time-limited presigned URLs — no public bucket needed.
- Default encryption: SSE-S3 (or SSE-KMS).
- Optional lifecycle rule: expire `backups/` objects after 90–365 days.
- Bucket CORS: not required (uploads go through the backend, not browser→S3).

8.5 CloudWatch

Optional for launch. PM2 logs + `pm2-logrotate` + a free uptime monitor (§23) cover the essentials. Add the CloudWatch agent later for disk/mem alarms if you stay on one box.

9. Domain / DNS Configuration

Only **one record** is required:

Type	Name	Value	TTL
A	`erp` (→ `erp.<school>.com`)	`<ELASTIC_IP>`	300

Optional: if you also want the apex or `www` to reach it, add a redirect at the registrar/CDN level, or a second A record + an Nginx `server` block that 301s to `https://erp.<school>.com`. Not required for the ERP itself.

Verify before requesting a certificate:

```
dig +short erp.<school>.com      # must return <ELASTIC_IP>
# or:  nslookup erp.<school>.com
```

> **WAIT FOR PROPAGATION**
> Certbot's HTTP-01 challenge will fail until `dig` returns the right IP from the server itself. Usually <5 min, allow up to 30.

10. Database Configuration (MongoDB Atlas)

1. Create a project + cluster (M0 to validate; M10+ for production traffic and backups).
2. **Database Access** → Add user `erpUser`, "Read and write to any database" (or scope to `school-erp`). Strong generated password.
3. **Network Access** → Add IP: `<ELASTIC_IP>/32`. Avoid `0.0.0.0/0`. (Add your office IP temporarily if you need Compass access.)
4. Connect → "Drivers" → copy the `mongodb+srv://…` string. Insert the DB name `school-erp` before the `?`: `mongodb+srv://erpUser:PASS@cluster0.abcde.mongodb.net/school-erp?retryWrites=true&w=majority`
5. TLS is on by default for `+srv`. The code sets pool 10–50 and a 5 s server-selection timeout — no extra options needed.
6. Enable Atlas automated backups (M10+) or schedule `mongodump` (§27).

> **FIRST CONNECTION = AUTO-SEED**
> On first successful connect the app seeds default classes (Nursery–10), the permission catalog, and the superadmin (per §5.3). The permission catalog re-syncs on every boot — expected, not a bug.

11. Backend Deployment

11.1 Base packages (run once)

```
ssh -i <SSH_KEY> <EC2_USER>@<ELASTIC_IP>

sudo apt update && sudo apt -y upgrade
sudo apt -y install git curl ufw nginx redis-server \
     build-essential python3 pkg-config ffmpeg libvips42

# Node 20 LTS (matches the Dockerfile)
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt -y install nodejs
node -v            # v20.x
sudo npm i -g pm2

sudo systemctl enable --now redis-server
redis-cli ping     # PONG
```

11.2 (t3.micro only) add swap

```
sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

11.3 Firewall

```
sudo ufw allow OpenSSH
sudo ufw allow 'Nginx Full'      # opens 80 + 443
sudo ufw --force enable
sudo ufw status
```

11.4 Get the code

```
sudo mkdir -p /var/www/school-erp && sudo chown -R $USER:$USER /var/www/school-erp
cd /var/www/school-erp
git clone https://github.com/team-IIT/rnd-service.git api
git clone https://github.com/team-IIT/rnd-ui.git      ui
cd api && git checkout main        # or the merged prod-hardening tag
```

11.5 Configure & build the API

```
cd /var/www/school-erp/api
cp .env.example .env
nano .env                          # fill per §7.2 — NODE_ENV=production
chmod 600 .env

npm ci                             # clean install from package-lock
npm run build                      # tsc → dist/  (dist/server.js, dist/worker.js)
ls dist/server.js dist/worker.js   # both must exist
```

11.6 First boot in the foreground (sanity check before PM2)

```
node dist/server.js
# Expect:
#   Connecting to MongoDB at: mongodb+srv://***:***@...
#   MongoDB Connected Successfully.
#   [WhatsApp Config] Live mode ...   (or simulation)
#   Server is running on port 5000
# In another SSH session:
curl -s localhost:5000/healthz        # {"status":"ok",...}
# Ctrl-C to stop.
```

IF IT EXITS WITH [FATAL]

Read the message — it is almost always a missing/weak `JWT_SECRET` / `JWT_REFRESH_SECRET` or a partial `WHATSAPP_*` block. Fix `.env` and retry. Do not proceed until `/healthz` returns `"status":"ok"`.

11.7 Start under PM2

```
cd /var/www/school-erp/api
pm2 start ecosystem.config.js --env production
pm2 status                            # school-erp-backend + school-erp-worker = online
pm2 logs --lines 50                   # clean, no crash loop
pm2 save
pm2 startup systemd -u $USER --hp /home/$USER
#   ^ run the exact `sudo env PATH=... pm2 startup ...` line it prints
pm2 save
```

11.8 Directory permissions

```
# the worker/api write only to os.tmpdir() and S3; nothing else on disk.
# Ensure the deploy user owns the tree:
sudo chown -R $USER:$USER /var/www/school-erp
```

11.9 Alternative: Docker Compose

The repo ships `Dockerfile` + `docker-compose.yml` (backend + redis + nginx). To use that path instead of PM2:

```
sudo apt -y install docker.io docker-compose-v2
# docker-compose.yml reads MONGODB_URI, JWT_*, AWS_*, WHATSAPP_*, APP_HOST,
# CORS_ALLOWED_ORIGINS from a sibling .env — create it (§7.2 minus REDIS_* which
# compose overrides to the `redis` service).
# Replace nginx/nginx.conf with a config equivalent to §13 (serving the SPA from a
# mounted volume and proxying to `backend:5000`), mount certs into nginx/certs.
docker compose up -d --build
docker compose ps
docker compose logs -f backend
```

Trade-off: reproducible image and no host Node/Redis, but you own cert renewal into the container and a heavier box. The rest of this guide assumes the PM2 path.

12. Frontend Deployment

```
cd /var/www/school-erp/ui
git checkout main
npm ci
printf 'VITE_API_URL=https://<DOMAIN>/api\n' > .env.production
npm run build                       # → dist/
ls dist/index.html dist/assets      # present

# publish to the path Nginx serves
sudo mkdir -p /var/www/school-erp/ui-dist
sudo rsync -a --delete dist/ /var/www/school-erp/ui-dist/
sudo chown -R www-data:www-data /var/www/school-erp/ui-dist
```

> **VERIFY THE API URL IS BAKED IN CORRECTLY**
>
> `grep -ro 'https://[a-z.]*/api' /var/www/school-erp/ui-dist/assets/*.js | head` — you should see `https://<DOMAIN>/api` and **no** `:5000` and **no** `localhost`. If you see `localhost`, `.env.production` was missing at build time — rebuild.

The SPA has no server-side routes. Its only "routes" are `/`, `/super-admin` (and alias `/superadmin`), plus `/?verify_token=…` and `/?report_verify_token=…`. All are served by `index.html` via the Nginx SPA fallback (§13/§17).

13. Nginx Configuration

Create `/etc/nginx/sites-available/school-erp`. This **replaces** the repo's `nginx/nginx.conf` (which is Docker-only and API-only).

```
server {
    listen 80;
    listen [::]:80;
    server_name <DOMAIN>;
    # ACME HTTP-01 challenge, then force HTTPS
    location /.well-known/acme-challenge/ { root /var/www/html; }
    location / { return 301 https://$host$request_uri; }
}

server {
    listen 443 ssl;
    listen [::]:443 ssl;
    http2 on;
    server_name <DOMAIN>;

    # ── TLS (Certbot fills these in / manages renewal) ──
    ssl_certificate     /etc/letsencrypt/live/<DOMAIN>/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/<DOMAIN>/privkey.pem;
    include             /etc/letsencrypt/options-ssl-nginx.conf;
    ssl_dhparam         /etc/letsencrypt/ssl-dhparams.pem;

    # ── Security headers (app has no helmet — set them here) ──
    add_header X-Frame-Options            SAMEORIGIN                      always;
    add_header X-Content-Type-Options     nosniff                         always;
    add_header Referrer-Policy            strict-origin-when-cross-origin   always;
    add_header Strict-Transport-Security  "max-age=31536000; includeSubDomains" always;

    # ── Body size: matches Express (§5.7). Raise both together if needed. ──
    client_max_body_size 60m;

    # ── SPA (static) ──
    root  /var/www/school-erp/ui-dist;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }
    # Hashed build assets — safe to cache hard
    location /assets/ {
        expires 1y;
        add_header Cache-Control "public, immutable";
        try_files $uri =404;
    }

    # ── API ──
    location /api/ {
        proxy_pass http://127.0.0.1:5000;
        proxy_http_version 1.1;
        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 120s;
        proxy_send_timeout 120s;
    }

    # ── Socket.IO (WebSocket upgrade) ──
    location /socket.io/ {
        proxy_pass http://127.0.0.1:5000;
        proxy_http_version 1.1;
```

```
        proxy_set_header Upgrade           $http_upgrade;
        proxy_set_header Connection        "upgrade";
        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 3600s;
    }

    # ── Health check (uptime monitors) ──
    location = /healthz {
        proxy_pass http://127.0.0.1:5000/healthz;
        access_log off;
    }

    # ── Never expose internal Prometheus metrics ──
    location = /metrics { return 404; }
}
```

Directive rationale

Directive	Why
`try_files $uri $uri/ /index.html`	SPA fallback: a browser refresh on `/super-admin` (or any path) returns `index.html`, not a 404, so client-side view selection works. §17.
`location /api/` before `location /`	Prefix match on `/api/` wins for API calls; everything else falls through to the SPA. Keeps API off the static handler.
`Upgrade` / `Connection "upgrade"` on `/socket.io/`	Required for the WebSocket handshake; without it Socket.IO silently downgrades to slow polling or fails.
`X-Forwarded-For` / `-Proto`	Feeds `app.set('trust proxy', 1)` so `req.ip` is the real client (login rate limiting) and `req.protocol` is `https`.
`proxy_read_timeout 120s` on `/api/`	Report-card / backup generation and large S3 round-trips exceed the 60 s default.
`client_max_body_size 60m`	Nginx rejects bodies over this with 413 *before* they reach Node. Must be ≥ the Express JSON limit.
`return 404` for `/metrics`	Cheapest fix for issue F — the endpoint is never reachable from outside.

Enable & test

```
sudo ln -s /etc/nginx/sites-available/school-erp /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t                       # syntax OK  (TLS lines will error until §14 — see note)
sudo systemctl reload nginx
```

> **ORDER**
>
> On a fresh box the `ssl_certificate` paths don't exist yet, so `nginx -t` fails. Two options: (a) temporarily comment the `server { listen 443 … }` block, reload, run Certbot (§14) with `--nginx` (it writes the TLS block for you), then paste back the extra `location` blocks; or (b) run `sudo certbot certonly --standalone -d <DOMAIN>` first (stop Nginx for 60 s), then this full config validates.

14. HTTPS / SSL Configuration

```
sudo snap install core && sudo snap refresh core
sudo snap install --classic certbot
sudo ln -sf /snap/bin/certbot /usr/bin/certbot

sudo certbot --nginx -d <DOMAIN> --redirect -m ops@<school>.com --agree-tos --no-eff-email

# Certbot edits the :80/:443 server blocks, installs the cert, and adds a
# systemd timer for renewal. Verify:
sudo systemctl list-timers | grep certbot
sudo certbot renew --dry-run          # must succeed
```

Verify TLS end to end

```
curl -I  http://<DOMAIN>              # 301 → https://
curl -sI https://<DOMAIN> | head      # 200, HSTS header present
curl -s  https://<DOMAIN>/healthz     # {"status":"ok",...}
# Browser: padlock, no "not secure", no mixed-content warnings in console.
# WebSocket: after login, DevTools → Network → WS shows 101 Switching Protocols (wss://).
```

> **AUTO-RENEW**
> The snap Certbot installs `certbot.timer` (twice daily). Renewal reloads Nginx automatically. Test yearly with `certbot renew --dry-run`.

Cookie / TLS interaction

The refresh cookie is set with `Secure` when `NODE_ENV=production`. It is therefore **only sent over HTTPS**. Any plain-HTTP testing of login will appear to "not persist the session" — that's expected; always test on `https://`.

15. PM2 / Process Management

Two apps are defined in `ecosystem.config.js`: `school-erp-backend` (`dist/server.js`) and `school-erp-worker` (`dist/worker.js`), both `fork` mode, 1 instance, restart at 1 GB RSS. Env is loaded by the app itself via `import 'dotenv/config'`, so `.env` must sit in the cwd where PM2 starts (the API repo root).

Command	Use
`pm2 start ecosystem.config.js --env production`	Initial start (sets `NODE_ENV=production`)
`pm2 status` / `pm2 ls`	Both processes `online`, low restart count
`pm2 logs school-erp-backend --lines 100`	Tail API logs
`pm2 logs school-erp-worker`	Tail BullMQ / scheduler logs
`pm2 restart school-erp-backend`	Hard restart (brief drop; fine — single instance)
`pm2 reload school-erp-backend`	Graceful; in fork mode ≈ restart. Use `restart` for clarity.
`pm2 save`	Persist the current process list (do after every start/stop change)
`pm2 startup systemd …`	Generate the boot service so PM2 + apps survive reboot
`pm2 install pm2-logrotate`	Cap log size (see below)

Log rotation

```
pm2 install pm2-logrotate
pm2 set pm2-logrotate:max_size 20M
pm2 set pm2-logrotate:retain 14
pm2 set pm2-logrotate:compress true
```

Survives:

- **SSH disconnect** — PM2 daemon is detached. ✓
- **App crash** — `uncaughtException` handler exits non-zero (server.ts:257); PM2 restarts it. ✓
- **Memory leak** — `max_memory_restart: '1G'`. ✓
- **Reboot** — `pm2 startup` + `pm2 save`. ✓ (verify with `sudo reboot` then `pm2 ls`.)

16. CORS Configuration

With single-domain path routing, browser calls to `/api` and `/socket.io` are **same-origin** — CORS is not triggered at all. The config still matters as defence in depth and for any future subdomain split.

HTTP CORS (src/server.ts:77-96) — already correct

- Allow-list = `['http://localhost:5173','http://localhost:5174']` + `CORS_ALLOWED_ORIGINS` (CSV) + any private-LAN IP.
- `credentials: true` — required for the refresh cookie.
- Unknown origins get `callback(null, false)` (no CORS headers) rather than an error — fine.
- **Action:** set `CORS_ALLOWED_ORIGINS=https://<DOMAIN>` in `.env`.

Socket.IO CORS — must be fixed (§5.2)

Currently `origin:'*'` + `credentials:true` (invalid). After §5.2 it mirrors the HTTP allow-list.

Optional hardening (issue N)

To stop production from trusting `localhost`/LAN origins, gate the dev entries:

```
const DEV_ORIGINS = process.env.NODE_ENV === 'production'
  ? [] : ['http://localhost:5173', 'http://localhost:5174'];
// and skip isPrivateLanOrigin() when NODE_ENV === 'production'
```

Preflight / methods

The `cors` package auto-handles `OPTIONS` preflight and reflects requested methods/headers (`GET/POST/PUT/PATCH/DELETE`, `Authorization`, `Content-Type`). No manual method list needed. Because routing is same-origin in production, you will normally see **no** `OPTIONS` requests in the Network tab at all — that's the sign it's set up right.

17. API Routing Configuration

Routing map (production)

Request path	Handled by	Notes
`/`, `/super-admin`, `/superadmin`, `/?verify_token=…`, `/?report_verify_token=…`, any other path	Nginx static → `index.html`	SPA reads `location.pathname` / query string and renders the right view.
`/api/*`	Nginx → `127.0.0.1:5000`	All REST endpoints. Public vs JWT-guarded is decided inside Express (server.ts:128-173).
`/socket.io/*`	Nginx → `127.0.0.1:5000` (WS upgrade)	Realtime messaging + school-profile push.
`/healthz`	Nginx → backend	Unauthenticated health JSON.
`/metrics`	Nginx → `404`	Blocked (issue F).

> **DO NOT PROXY `/SUPER-ADMIN` TO THE BACKEND**
>
> The backend has `GET /superadmin` and `/super-admin` that 301-redirect to `${APP_HOST}/super-admin`. If Nginx forwarded `/super-admin` to Node, Node would redirect to `https://<DOMAIN>/super-admin` → Nginx → Node → **redirect loop**. The config in §13 correctly lets the SPA fallback serve both paths; the backend redirect route is simply shadowed and unused. That is intended.

Backend API prefixes (reference)

```
Public:   /api/auth/*        /api/receipts       /api/whatsapp/webhook
          /api/whatsapp/health(auth)  /api/school-profile  /api/report-card/verify
Guarded:  /api/teachers  /api/students  /api/dashboard  /api/events  /api/settings
          /api/classes   /api/fee-structures  /api/expenditure  /api/homework
          /api/attendance  /api/marks  /api/leaves  /api/delegations  /api/feedbacks
          /api/parent/profile  /api/messages
Login rate limit: /api/auth/login  → 20 / 15 min / IP  (Redis-backed)
```

18. Authentication Configuration

18.1 Flow

```
POST /api/auth/login  (username|email|employeeId|phone + password)
   → 200 { accessToken (JWT, 1h), user{...} }
   → Set-Cookie: refreshToken=<JWT 7d>; HttpOnly; Secure; SameSite=Strict; Max-Age=604800
SPA keeps accessToken in memory only; sends `Authorization: Bearer` on every /api call.
On 401 → POST /api/auth/refresh (cookie only) → new accessToken + rotated cookie.
Session store: Redis key `session:refresh:<uid>:<sha256(token)>` (TTL 7d), Mongo fallback.
Logout → POST /api/auth/logout → deletes session + clears cookie.
```

18.2 Why single-domain routing is mandatory here

> **SAMESITE=STRICT + CROSS-SITE FETCH = BROKEN LOGIN**
> The refresh cookie is `SameSite=Strict` (auth.controller.ts:121). The SPA calls `POST /api/auth/refresh` with `fetch(..., {credentials:'include'})`. If the SPA were on `app.school.com` and the API on `api.school.com`, that fetch is *cross-site* and the browser would **not send the cookie** — every session would silently fail to refresh and users would be logged out after 1 hour. Serving both from `https://<DOMAIN>` makes the request same-site and the cookie flows. If you ever must split subdomains, you must change the cookie to `SameSite=None; Secure` and set an explicit parent `Domain` — a code change, not a config one.

18.3 Behind Nginx — checklist

- `app.set('trust proxy', 1)` applied (§5.1) → `req.ip` correct, `Secure` cookie honoured over the proxy, rate-limit per real IP.
- `X-Forwarded-Proto: https` forwarded (§13) → `secure` cookie logic + any `req.protocol` checks see HTTPS.
- Cookie `HttpOnly` → not readable by JS (XSS can't steal the refresh token). ✓ already.
- Access token in memory (not `localStorage`) → lost on tab close, re-obtained via refresh cookie. ✓ already.
- JWT secrets are 64-hex random and differ → enforced at boot in production. ✓ via §7.2.

18.4 First-login superadmin hardening (issue D)

```
# After the first successful boot, log in at https://<DOMAIN>/super-admin
#   username: superadmin
#   password: <SUPERADMIN_INITIAL_PASSWORD>   (or 123 if you did not apply §5.3)
# Immediately: Profile → Change Password  → set a strong unique password.
# Then remove SUPERADMIN_INITIAL_PASSWORD from .env and: pm2 restart school-erp-backend
# Verify no stray admin/superadmin accounts:  (Mongo)  db.users.find({role:{$in:
['admin','superadmin']}})
```

18.5 Role model

`superadmin` (bypasses all role checks), `admin`, `teacher`, `other` (custom employee, permission-gated), `parent`. Fine-grained permissions come from a PBAC catalog resolved per request (services/permissionEngine.ts), cached in Redis for 10 min (config/passport.ts:96-129) — so permission changes take up to 10 min to propagate, or clear `user:profile:*` keys.

19. File Upload Configuration

Path	Behaviour
Student/teacher docs, event media, receipts, report cards, message attachments	Sent as base64 in JSON → uploaded to **S3** → served via presigned GET URLs (default 1 h).
Message attachments < 10 MB, only when `AWS_BUCKET_NAME` is unset	Written to local `uploads/`, served at `/uploads` (Express static). **Ephemeral — lost on redeploy.**
Voice notes	webm → ogg/opus via `ffmpeg-static` in `os.tmpdir()`, then S3. Requires `ffmpeg` deps (installed in §11.1).
Backups	Student CSV → S3 `backups/students/…`. **Hard-fails if `AWS_BUCKET_NAME` unset** (config/worker.ts:19).

> **PRODUCTION REQUIREMENT**
> Set `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` / `AWS_REGION` / `AWS_BUCKET_NAME`. Do not rely on the local fallback. If you keep it as a safety net, add `uploads/` to a persistent path and back it up — but S3 is the supported path.

Size limits — keep these three in sync

Layer	Setting	File
Nginx	`client_max_body_size 60m`	§13
Express JSON	`limit: '60mb'` (was `250mb` — §5.7)	server.ts:100
App logic	> 10 MB message attachments forced to S3	message.routes.ts:34

If a legitimate upload (e.g. a large event gallery) needs more, raise the Nginx and Express numbers together and restart both.

20. WhatsApp Cloud API & Webhook

20.1 Modes

- **Simulation** — *no* `WHATSAPP_*` vars set. Messages are logged, not sent. Fine for a soft launch.
- **Live** — all required vars valid. A *partial* config is a hard boot error.

20.2 Required vars (live)

```
WHATSAPP_ACCESS_TOKEN        # permanent System-User token (≥50 chars)
WHATSAPP_PHONE_NUMBER_ID     # numeric
WHATSAPP_BUSINESS_ACCOUNT_ID # numeric
WHATSAPP_WEBHOOK_VERIFY_TOKEN # you invent; must match Meta dashboard
WHATSAPP_APP_SECRET          # Meta App → Settings → Basic → App Secret
```

20.3 Configure the webhook in Meta

Field	Value
Callback URL	`https://<DOMAIN>/api/whatsapp/webhook`
Verify token	= `WHATSAPP_WEBHOOK_VERIFY_TOKEN`
Subscribe fields	`messages` (delivery statuses + inbound replies)

> **ISSUE K — APP SECRET IS MANDATORY IN PRODUCTION**
> Without `WHATSAPP_APP_SECRET`, whatsappWebhook.routes.ts:57-68 **rejects every inbound delivery with 401** when `NODE_ENV=production`. Delivery receipts and parent replies will silently stop. Set it.

20.4 Verify

```
curl -s "https://<DOMAIN>/api/whatsapp/webhook?hub.mode=subscribe&hub.verify_token=
<TOKEN>&hub.challenge=123"
#   → 123     (echoes the challenge = success)
# Authenticated deep check (needs a superadmin bearer token + whatsapp:view):
curl -s https://<DOMAIN>/api/whatsapp/health -H "Authorization: Bearer <JWT>"
# /healthz also reports  "whatsapp":"live"  or  "simulation"
```

The old dev flow (`start_all.bat` → ngrok tunnel `stimulate-prize-unwanted.ngrok-free.dev`) is **not used** in production — the public HTTPS domain is the callback.

21. Security Hardening

Area	Status / Action
Secrets in git	`.env` never committed (verified). Apply §5.5 so `.env.production` can't be either. Dev JWT fallbacks in config/secrets.ts are blocked at boot in production.
JWT secrets	64-hex random, distinct, env-only. Rotating them invalidates all sessions (acceptable during a maintenance window).
Transport	TLS 1.2/1.3 via Certbot; HSTS header (§13); HTTP 301→HTTPS.
HTTP security headers	Set in Nginx (§13): `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`, `Strict-Transport-Security`. Optionally add `helmet` to Express later.
Rate limiting	Login: 20/15min/IP (needs §5.1 to be per-IP). Consider a global `/api` limiter (e.g. 300/min/IP) as a later hardening.
CORS	Allow-list only (§16); fix Socket.IO `*` (§5.2).
Auth / RBAC	Passport JWT on all non-public routes; role + PBAC checks. Superadmin bootstrap hardened (§5.3/§18.4).
Webhook	HMAC signature verification via `WHATSAPP_APP_SECRET` (§20.3).
File uploads	To private S3; presigned URLs expire; body size capped at Nginx + Express.
Request size	60 MB cap (§5.7/§19) — mitigates base64 memory-exhaustion.
Error leakage	Global handler omits stack traces when `NODE_ENV=production` (server.ts:271). OTP values are not logged in production (auth.controller.ts:451). Keep `NODE_ENV=production`.
Internal endpoints	`/metrics` blocked at Nginx (§13). `/healthz` is safe to expose (no secrets).
Public ports	Only 80/443. Node `:5000` and Redis `:6379` bind localhost; SG blocks them anyway.
SSH	Key-only, port 22 from your IP /32. Consider `fail2ban` and disabling password auth (`sshd_config`: `PasswordAuthentication no`).
Redis	Bound to `127.0.0.1` by default on Ubuntu. Confirm `bind 127.0.0.1 ::1` in `/etc/redis/redis.conf`. Optionally set `requirepass`.
OS updates	`sudo apt install unattended-upgrades` for security patches.
Dependency audit	`npm audit --production` in both repos before each release; patch highs.

22. Health Checks

The app already exposes one (no new endpoint needed):

```
GET /healthz          (unauthenticated)
200 {
  "status": "ok" | "degraded",     // "degraded" when Mongo is not connected
  "uptimeSeconds": 1234,
  "mongo": "connected" | "disconnected",
  "redis": "connected" | "disconnected",
  "s3": "configured" | "not-configured",
  "whatsapp": "live" | "simulation",
  "timestamp": "..."
}
```

ISSUE O

`/healthz` returns HTTP **200 even when** `status:"degraded"`. Configure your uptime monitor to alert on the JSON body (`status != "ok"` or `mongo != "connected"`), not just the status code.

Tests

```
curl -s https://<DOMAIN>/healthz | python3 -m json.tool
# From the box, bypassing Nginx:
curl -s localhost:5000/healthz
# Redis:
redis-cli ping
# Mongo (from the box):
node -e "require('mongoose').connect(process.env.MONGODB_URI).then(()=>
{console.log('ok');process.exit(0)}).catch(e=>{console.error(e.message);process.exit(1)})"
```

Recommended external monitor

UptimeRobot / BetterStack / Healthchecks.io hitting `https://<DOMAIN>/healthz` every 1–5 min with a keyword check for `"status": "ok"`.

23. Logging & Monitoring

Source	Command
API	`pm2 logs school-erp-backend` · files in `~/.pm2/logs/school-erp-backend-{out,error}.log`
Worker / scheduler	`pm2 logs school-erp-worker`
Nginx access	`sudo tail -f /var/log/nginx/access.log`
Nginx error	`sudo tail -f /var/log/nginx/error.log`
Redis	`sudo journalctl -u redis-server -f`
System / OOM	`sudo journalctl -k | grep -i oom` · `dmesg -T`
Prometheus	`curl -s localhost:5000/metrics` (from the box only — blocked externally). Point a private Prometheus/Grafana here later if desired.

Diagnosing common signals

Symptom	First look
`502 Bad Gateway`	`pm2 status` (is backend online?), `pm2 logs`, `curl localhost:5000/healthz`
`404` on browser refresh of a deep link	Nginx `try_files … /index.html` present? `sudo nginx -T | grep try_files`
`413` on upload	Nginx `client_max_body_size` vs Express limit
CORS error in console	Should not happen with same-origin routing. If seen: request is going to `:5000` → `VITE_API_URL` wrong; rebuild frontend (§12).
Mixed-content warning	An `http://` link — apply §5.4; check `APP_HOST` has no scheme.
`ECONNREFUSED 127.0.0.1:6379`	Redis down → `sudo systemctl restart redis-server`. Login will 500 until it's back.
Mongo connection failure	Atlas Network Access missing the Elastic IP; or `MONGODB_URI` password wrong. `pm2 logs` shows the redacted URI.
JWT failures after deploy	`JWT_SECRET` changed → all tokens invalid (expected); users just re-login.
WebSocket won't connect	Nginx `/socket.io/` block missing `Upgrade` / `Connection` headers; or auth token expired.

24. Production Testing Checklist

Infrastructure

- ☐ `ssh` in works; `ufw status` shows only 22/80/443
- ☐ `dig +short <DOMAIN>` → Elastic IP
- ☐ `curl -I http://<DOMAIN>` → 301 to https
- ☐ `curl -sI https://<DOMAIN>` → 200 + HSTS; browser padlock valid
- ☐ `sudo certbot renew --dry-run` passes
- ☐ `sudo reboot` → after ~1 min, `pm2 ls` both online, site up

Frontend

- ☐ `https://<DOMAIN>/` loads the login page; assets 200; no console errors
- ☐ Refresh on `/super-admin` → still loads (no Nginx 404)
- ☐ Network tab: API calls go to `https://<DOMAIN>/api/...` (no `:5000`, no `localhost`, no mixed content)

Backend

- ☐ `curl https://<DOMAIN>/healthz` → `status:"ok"`, `mongo:"connected"`, `redis:"connected"`
- ☐ `s3:"configured"`, `whatsapp:"live"` (or intentionally `simulation`)
- ☐ `pm2 logs` — no crash loop, no unhandled rejection spam

Authentication

- ☐ superadmin login at `/super-admin`; forced password change completes (§18.4)
- ☐ admin login; refresh cookie present (DevTools → Application → Cookies): `HttpOnly`, `Secure`, `SameSite=Strict`
- ☐ Leave the tab 1 h+ (or delete the in-memory token) → next API call silently refreshes, no logout
- ☐ Logout clears the cookie; protected route now returns 401
- ☐ 21 rapid bad logins from one client → "Too many login attempts" (rate limiter, per-IP)

ERP modules (only those present)

- ☐ Students: list, create, edit, photo upload → renders from S3 URL
- ☐ Teachers: list, create, signature upload (background removal runs)
- ☐ Classes / Fee structures: load and edit
- ☐ Attendance (teacher): mark + save
- ☐ Marks / Exams: enter marks; Report Card generates (PDF) and its verify link opens `https://<DOMAIN>/?report_verify_token=…`
- ☐ Fees: collect a payment → receipt PDF to S3 + WhatsApp message with `https://` verify link
- ☐ Expenditure, Leaves, Delegations, Feedback, Homework, Events: load
- ☐ Messages: send a message → recipient's browser gets it in realtime (Socket.IO); WhatsApp attachment >10 MB goes to S3
- ☐ Admin Settings → Backup: triggers a job; worker uploads CSV to `s3://<bucket>/backups/students/…`
- ☐ Parent portal: login with admission no; results, notices, feedback, profile
- ☐ Super Admin portal: collection explorer, queue monitor, redis inspector load

WhatsApp

- ☐ Meta webhook shows "Verified"; `hub.challenge` curl echoes
- ☐ A real fee receipt / OTP message is delivered to a test number
- ☐ Delivery status flows back (webhook 200, not 401 → App Secret set)

25. Troubleshooting Guide

Problem	Likely cause	Diagnose	Fix
App calls `http://…:5000/api`, all requests fail	`VITE_API_URL` missing at build time	DevTools → Network; `grep :5000 ui-dist/assets/*.js`	Create `.env.production` (§7.3), `npm run build`, rsync, hard-refresh
CORS error in console	Not actually same-origin (points at `:5000` or another host)	Compare request Origin vs host	Fix `VITE_API_URL`; set `CORS_ALLOWED_ORIGINS`; apply §5.2
`502 Bad Gateway`	Backend down / crashed / wrong port	`pm2 status`; `pm2 logs`; `curl localhost:5000/healthz`	`pm2 restart school-erp-backend`; fix the logged error (usually `.env`)
`404` after refreshing `/super-admin`	Missing SPA fallback	`sudo nginx -T | grep -A2 'location /'`	Add `try_files $uri $uri/ /index.html`; reload Nginx
Redirect loop on `/super-admin`	Nginx proxies it to the backend redirect route	`curl -I https://<DOMAIN>/super-admin` shows repeated 301	Remove any `location /super-admin` proxy; let the SPA fallback serve it (§17)
Login succeeds then user is logged out ~1h later	Refresh cookie not sent (cross-site) or `Secure` over HTTP	DevTools → Application → Cookies; check request to `/api/auth/refresh`	Use single domain (§18.2); ensure HTTPS + `trust proxy` (§5.1)
Everyone locked out of login at once	`trust proxy` not set → all traffic looks like one IP	Rate-limit triggers globally; `pm2 logs` shows express-rate-limit validation warning	Apply §5.1; `pm2 restart`; clear the Redis `rl:` keys
`500` on every login	Redis down (rate-limit store)	`redis-cli ping`; `pm2 logs` → `ECONNREFUSED :6379`	`sudo systemctl restart redis-server`
`413 Request Entity Too Large` on upload	Body > `client_max_body_size`	Nginx error log	Raise Nginx + Express limits together (§19); or push the file to S3 path
Mongo: "could not connect / server selection timeout"	Atlas IP allow-list / bad password	`pm2 logs` (redacted URI); Atlas → Network Access	Add `<ELASTIC_IP>/32`; fix `MONGODB_URI`
Cert renewal fails	Port 80 blocked / DNS changed	`sudo certbot renew --dry-run`; `sudo ss -tlnp | grep :80`	Open 80 in SG + UFW; confirm A record
Mixed-content / insecure link in WhatsApp msg	`http://` hardcoded (issue E)	Inspect the sent message text	Apply §5.4; set `APP_HOST=<DOMAIN>` (no scheme)
WebSocket fails / falls back to polling	Nginx upgrade headers missing	DevTools → Network → WS (no 101)	Add the `/socket.io/` block (§13); reload Nginx
WhatsApp webhook returns 401	`WHATSAPP_APP_SECRET` unset in production	`pm2 logs` → "rejecting all webhook deliveries"	Set the App Secret; `pm2 restart`
Server won't boot, `[FATAL]`	Weak/missing JWT secret or partial WhatsApp config	Read the fatal message	Fix `.env` per §7.2

Box gets slow / OOM-kills node	250 MB body limit + small RAM	`journalctl -k | grep -i oom` ; `free -m`	Apply §5.7; add swap (§11.2); upsize to `t3.small`
Signature/background-removal errors	Missing native libs	`pm2 logs` → sharp/vips/onnx error	`sudo apt install libvips42 ffmpeg build-essential python3` ; `npm ci` ; rebuild

26. Rollback Procedure

Every deploy is a git checkout + build + `pm2 restart`. Rollback is the same in reverse. Keep the previous commit SHA before you deploy.

26.1 Backend rollback

```
cd /var/www/school-erp/api
git log --oneline -5                    # find last-good SHA
git checkout <LAST_GOOD_SHA>
npm ci                                   # only if package-lock changed
npm run build
pm2 restart school-erp-backend school-erp-worker
curl -s https://<DOMAIN>/healthz         # status:"ok"
```

26.2 Frontend rollback

```
# Option A: keep timestamped release dirs (recommended, see §28)
sudo ln -sfn /var/www/school-erp/releases/ui-<PREV_TS> /var/www/school-erp/ui-dist
sudo systemctl reload nginx
# Option B: rebuild from the previous SHA
cd /var/www/school-erp/ui && git checkout <LAST_GOOD_SHA> && npm ci && npm run build
sudo rsync -a --delete dist/ /var/www/school-erp/ui-dist/
```

26.3 Config rollback

```
sudo cp /etc/nginx/sites-available/school-erp.bak /etc/nginx/sites-available/school-erp
sudo nginx -t && sudo systemctl reload nginx
# .env: restore from your secrets vault / previous backup, then pm2 restart
```

26.4 Database

> **SCHEMA NOTE**
>
> There are no destructive migrations — `connectDB` only does *additive* seeding (classes, permission catalog, a one-time `transportFee` backfill). A code rollback needs no DB rollback. If you must restore data, use the Atlas point-in-time restore or your latest `mongodump` (§27) — this replaces data, so take a fresh dump first.

26.5 Decision guide

Failure	Action
Health check red immediately after deploy	Roll back backend (26.1). Investigate on a branch.
Frontend broken, API fine	Roll back frontend only (26.2 Option A — seconds).
Nginx test fails	You never reloaded a broken config (`nginx -t` gates it). Fix forward.
Bad `.env` value	Fix the value, `pm2 restart`. No code rollback needed.

27. Backup & Recovery

Asset	Method	Frequency	Location
MongoDB data	Atlas automated backups (M10+) *or* cron `mongodump --uri "$MONGODB_URI" --archive --gzip` → S3	Daily, retain 14–30	Atlas / `s3://<bucket>/db-backups/`
App data exports	Built-in student CSV backup job (Admin → Settings → Backup)	On demand / weekly	`s3://<bucket>/backups/students/`
Uploaded files	Already in S3. Enable **bucket versioning** + optionally cross-region replication	Continuous	S3
Backend `.env`	Store in a password manager / AWS Secrets Manager / SSM Parameter Store. Never on disk-only.	On every change	Secrets vault
Nginx config	`sudo cp … school-erp.bak` before edits; also commit a copy to the infra repo	On every change	Git + box
TLS certs	Certbot re-issues automatically; nothing to back up. Keep `/etc/letsencrypt/` in the AMI snapshot.	—	—
Whole server	AWS → EC2 → Create Image (AMI) snapshot	After major changes	AWS AMI

Nightly DB dump (cron on the box)

```
sudo apt -y install mongodb-database-tools awscli
mkdir -p /var/backups/erp
cat <<'EOF' | sudo tee /usr/local/bin/erp-db-backup.sh
#!/bin/bash
set -euo pipefail
source /var/www/school-erp/api/.env
TS=$(date +%F-%H%M)
OUT=/var/backups/erp/erp-$TS.archive.gz
mongodump --uri "$MONGODB_URI" --archive="$OUT" --gzip
aws s3 cp "$OUT" "s3://$AWS_BUCKET_NAME/db-backups/" --region "$AWS_REGION"
find /var/backups/erp -name 'erp-*.archive.gz' -mtime +7 -delete
EOF
sudo chmod +x /usr/local/bin/erp-db-backup.sh
( sudo crontab -l 2>/dev/null; echo "30 1 * * * /usr/local/bin/erp-db-backup.sh >> /var/log/erp-backup.log 2>&1" ) | sudo crontab -
```

Restore drill

```
# To a scratch database first — never straight over production:
mongorestore --uri "$MONGODB_URI_SCRATCH" --archive=erp-<TS>.archive.gz --gzip
# Validate, then repoint MONGODB_URI or restore in place with --drop.
```

Full-server recovery (worst case)

1. Launch a new EC2 from the latest AMI (or a fresh Ubuntu box + §11).
2. Associate the Elastic IP (DNS unchanged).

3. Restore `.env` from the vault; `git pull` both repos to the release tag.
4. `npm ci && npm run build` (both), rsync frontend, `pm2 start … && pm2 save`.
5. `sudo certbot --nginx -d <DOMAIN>` (or restore `/etc/letsencrypt`).
6. Run §24 checklist.

28. Future Deployment / Update Procedure

Put this `deploy.sh` at `/var/www/school-erp/deploy.sh`. One command, with a health gate and auto-rollback of the frontend.

```
#!/usr/bin/env bash
set -euo pipefail
ROOT=/var/www/school-erp
API=$ROOT/api
UI=$ROOT/ui
REL=$ROOT/releases
DOMAIN="<DOMAIN>"
TS=$(date +%Y%m%d-%H%M%S)
mkdir -p "$REL"

echo "▶ 1/7  Pull"
git -C "$API" pull --ff-only
git -C "$UI"  pull --ff-only
API_PREV=$(git -C "$API" rev-parse HEAD@{1} 2>/dev/null || git -C "$API" rev-parse HEAD)

echo "▶ 2/7  Backend deps + build"
( cd "$API" && npm ci && npm run build )

echo "▶ 3/7  Frontend build"
( cd "$UI" && npm ci && npm run build )
cp -r "$UI/dist" "$REL/ui-$TS"

echo "▶ 4/7  Swap frontend (atomic symlink)"
ln -sfn "$REL/ui-$TS" "$ROOT/ui-dist"
sudo systemctl reload nginx

echo "▶ 5/7  Restart backend"
pm2 restart school-erp-backend school-erp-worker --update-env
pm2 save

echo "▶ 6/7  Health gate"
sleep 4
if ! curl -fsS "https://$DOMAIN/healthz" | grep -q '"status": *"ok"'; then
  echo "✕ Health check FAILED — rolling back"
  git -C "$API" checkout "$API_PREV"
  ( cd "$API" && npm ci && npm run build )
  pm2 restart school-erp-backend school-erp-worker
  PREV_UI=$(ls -dt "$REL"/ui-* | sed -n 2p || true)
  [ -n "$PREV_UI" ] && ln -sfn "$PREV_UI" "$ROOT/ui-dist" && sudo systemctl reload nginx
  exit 1
fi

echo "▶ 7/7  Prune old releases (keep 5)"
ls -dt "$REL"/ui-* | tail -n +6 | xargs -r rm -rf
echo "✓ Deployed $TS"
```

```
chmod +x /var/www/school-erp/deploy.sh
# Routine deploy:
cd /var/www/school-erp && ./deploy.sh
```

> **PRE-FLIGHT (BEFORE RUNNING DEPLOY.SH)**
>
> `git -C api status` / `git -C ui status` clean · CI green on `main` · `npm audit --production` reviewed · `.env` unchanged or updated · any DB-affecting change reviewed (there are no migrations, but new required env vars need adding first).

29. Multi-School Deployment Considerations

The codebase today is effectively **single-tenant**: one `SchoolProfile` document, one WhatsApp Business Account from env, superadmin seeded per database. `config/whatsapp.ts` exposes a `resolveTenantConfig(schoolId)` seam for future per-school WABAs, but nothing else is tenant-aware.

	Independent deployment (recommended now)	**Shared multi-tenant (later, needs code)**
Isolation	Total — separate DB, S3 prefix, domain, process set, WhatsApp number	Logical only — `schoolId` on every document + every query
Blast radius	One school's outage/bug/データ never touches another	A bad query or migration hits everyone
Cost	~1 small VM + 1 Atlas cluster per school (or several schools per VM with separate PM2 apps + ports)	One larger cluster amortised
Onboarding a school	Repeat §7–§15 with a new `.env`, subdomain, DB, port	Insert a tenant row; no infra
Effort	Zero code change	Weeks: tenant middleware, per-tenant auth scoping, data migration, per-tenant seeds, WABA routing, noisy-neighbour controls

Recommended pattern for the next few schools — one box, many apps

```
/var/www/school-A/{api,ui,ui-dist}   PORT=5001   erp.school-a.com   DB school-erp-a   s3 prefix a/
/var/www/school-B/{api,ui,ui-dist}   PORT=5002   erp.school-b.com   DB school-erp-b   s3 prefix b/
# One Nginx server{} block per domain, each proxying to its 127.0.0.1:PORT
# PM2: school-A-backend, school-A-worker, school-B-backend, ...
# Each app has its own .env (own JWT secrets, own WABA, own APP_HOST/CORS_ALLOWED_ORIGINS)
```

Move to true multi-tenancy only when the per-school operational overhead (dozens of PM2 apps, dozens of certs) outweighs the isolation benefit — realistically past ~10–15 schools.

> **BLOCKER FOR SHARED MULTI-TENANT**
>
> The auto-seeded superadmin and single `SchoolProfile` assumption in `config/db.ts` must be reworked before any shared deployment.

30. Future CI/CD Automation

Not required now — `deploy.sh` (§28) is enough for a single box. When you want push-to-deploy:

```
GitHub push to main
      │
      ▼
GitHub Actions
  ├─ job: backend   → npm ci → npm run build → npm test (vitest)
  ├─ job: frontend  → npm ci → npm run build → (oxlint)
  └─ job: deploy (needs both, main only)
        → ssh <EC2> 'cd /var/www/school-erp && ./deploy.sh'
        → deploy.sh already does the health gate + rollback
      │
      ▼
Post: curl https://<DOMAIN>/healthz  → notify Slack on failure
```

Minimal workflow

```yaml
name: deploy
on: { push: { branches: [main] } }
jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm }
      - run: npm ci
      - run: npm run build
      - run: npm test --if-present
  deploy:
    needs: ci
    runs-on: ubuntu-latest
    steps:
      - uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.EC2_HOST }}
          username: ubuntu
          key: ${{ secrets.EC2_SSH_KEY }}
          script: cd /var/www/school-erp && ./deploy.sh
```

Add a second workflow for `rnd-ui` (separate repo) or make `deploy.sh` the single source of truth called by both. Store `EC2_HOST`, `EC2_SSH_KEY` as repo secrets. Keep the box's `.env` out of CI — it stays on the server / in SSM.

31. One-Page Deployment Runbook

> **PLACEHOLDERS**
> `<DOMAIN>` `<ELASTIC_IP>` `<SSH_KEY>` — replace before running.

PHASE 1 — PREPARE (LOCAL)

```
openssl rand -hex 32   # JWT_SECRET
openssl rand -hex 32   # JWT_REFRESH_SECRET (different)
# Apply code changes §5.1–§5.7 on a branch → build → test → merge to main
```

PHASE 2 — SERVER

```
ssh -i <SSH_KEY> ubuntu@<ELASTIC_IP>
sudo apt update && sudo apt -y upgrade
sudo apt -y install git curl ufw nginx redis-server build-essential python3 pkg-config ffmpeg
libvips42
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash - && sudo apt -y install nodejs
sudo npm i -g pm2 && sudo systemctl enable --now redis-server
sudo ufw allow OpenSSH && sudo ufw allow 'Nginx Full' && sudo ufw --force enable
```

PHASE 3 — ENVIRONMENT

```
sudo mkdir -p /var/www/school-erp && sudo chown -R $USER:$USER /var/www/school-erp
cd /var/www/school-erp
git clone https://github.com/team-IIT/rnd-service.git api
git clone https://github.com/team-IIT/rnd-ui.git      ui
cp api/.env.example api/.env && nano api/.env      # §7.2 · NODE_ENV=production · chmod 600
printf 'VITE_API_URL=https://<DOMAIN>/api\n' > ui/.env.production
```

PHASE 4 — DATABASE

```
# Atlas: create user erpUser, allow-list <ELASTIC_IP>/32, paste SRV string into MONGODB_URI
```

PHASE 5 — BACKEND

```
cd /var/www/school-erp/api && npm ci && npm run build
node dist/server.js   # sanity: "MongoDB Connected", "Server is running on port 5000" → Ctrl-C
pm2 start ecosystem.config.js --env production && pm2 save
pm2 startup systemd -u $USER --hp /home/$USER   # run the printed sudo line, then: pm2 save
pm2 install pm2-logrotate
```

PHASE 6 — FRONTEND

```
cd /var/www/school-erp/ui && npm ci && npm run build
sudo mkdir -p /var/www/school-erp/ui-dist
sudo rsync -a --delete dist/ /var/www/school-erp/ui-dist/
grep -ro 'https://[a-z.]*/api' ui-dist/assets/*.js | head   # must be https://<DOMAIN>/api
```

PHASE 7 — NGINX

```
sudo nano /etc/nginx/sites-available/school-erp     # paste §13 (comment 443 block for now)
sudo ln -s /etc/nginx/sites-available/school-erp /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl reload nginx
```

PHASE 8 — HTTPS

```
sudo snap install --classic certbot && sudo ln -sf /snap/bin/certbot /usr/bin/certbot
sudo certbot --nginx -d <DOMAIN> --redirect -m ops@<school>.com --agree-tos --no-eff-email
# uncomment/paste the extra location blocks from §13 into the 443 server → nginx -t → reload
sudo certbot renew --dry-run
```

PHASE 9 — DNS

```
# Registrar:  A  erp  →  <ELASTIC_IP>   (TTL 300)
dig +short <DOMAIN>      # == <ELASTIC_IP> before running certbot
```

PHASE 10 — START & WIRE EXTERNALS

```
pm2 restart school-erp-backend school-erp-worker
# Meta: webhook → https://<DOMAIN>/api/whatsapp/webhook, verify token =
WHATSAPP_WEBHOOK_VERIFY_TOKEN, field: messages
```

PHASE 11 — TEST

```
curl -I  http://<DOMAIN>                      # 301
curl -s  https://<DOMAIN>/healthz             # status:"ok", mongo/redis connected
# Browser: load site, login as superadmin at /super-admin, change password, refresh /super-admin
(no 404)
```

PHASE 12 — PRODUCTION VERIFICATION

```
# Run the full §24 checklist. Then:
sudo reboot        # after ~1 min:  pm2 ls (both online) + site loads
# Remove SUPERADMIN_INITIAL_PASSWORD from .env → pm2 restart school-erp-backend
```

32. Running List of Changes

This guide modified **no application code**. The following are to be applied by the operator during deployment.

Code (on a branch, §5)

1. rnd-service/src/server.ts — add `app.set('trust proxy', 1)`
2. rnd-service/src/config/socket.ts — Socket.IO CORS allow-list instead of `origin:'*'`
3. rnd-service/src/config/db.ts — superadmin seed reads `SUPERADMIN_INITIAL_PASSWORD` + `mustChangePassword:true`
4. rnd-service/src/routes/admin/dashboard.routes.ts, src/services/reportCard.service.ts, src/server.ts — verify links / redirect use `https://`
5. rnd-service/.gitignore — `.env.*` + `!.env.example`
6. rnd-service/src/server.ts — (optional) JSON limit 250mb → 60mb
7. rnd-service/src/server.ts — (optional) auth-gate `/metrics` if not blocking in Nginx

New files on the server

1. `/var/www/school-erp/api/.env` (§7.2)
2. `/var/www/school-erp/ui/.env.production` (§7.3)
3. `/etc/nginx/sites-available/school-erp` (§13)
4. `/var/www/school-erp/deploy.sh` (§28)
5. `/usr/local/bin/erp-db-backup.sh` + cron (§27)
6. TLS certs under `/etc/letsencrypt/` (Certbot, §14)
7. PM2 systemd unit (`pm2 startup` , §15)

External configuration

1. DNS A record `erp → <ELASTIC_IP>`
2. AWS: EC2 + Elastic IP + Security Group (22/80/443) + IAM S3 user + private S3 bucket
3. MongoDB Atlas: DB user + Network Access allow-list
4. Meta WhatsApp: webhook callback URL + verify token + `messages` subscription
5. Uptime monitor on `/healthz` (keyword `"status": "ok"`)

End of guide. Do not consider the deployment complete until every box in §24 is checked and a post-deploy `sudo reboot` leaves the site serving on `https://<DOMAIN>` .